

顔の共通性診断アプリ

Face Similarity Quantifier — プロジェクト引継ぎ資料

☑ テスト結果: 22件 / 22件 全パス

1. プロジェクト概要

本アプリケーションは複数の顔画像を入力として受け取り、顔の共通性（類似度）を定量的なスコアとして出力する StandaloneのPython製Webアプリです。既存のInstagramDownloader iOSアプリとは完全に独立した構成になっています。

2. 技術スタックと設計判断

役割	採用技術	選定理由
顔検出	OpenCV YuNet (ONNX)	軽量(1MB)・コンパイル不要・高精度
顔特徴量抽出	OpenCV SFace (ONNX)	128次元埋め込み・ArcFace損失・37MB
類似度計算	コサイン類似度 → 0~100%	スケール不変・直感的な百分率表示
Web UI	Streamlit	最小コードで対話的UIを実現
テスト	pytest (22件)	純粋数学層を優先的にカバー

3. ファイル構成

FaceSimilarity/ (リポジトリ内の独立ディレクトリ)

ファイル	役割
face_similarity.py	コアロジック: モデル管理・顔検出・埋め込み・類似度計算
app.py	Streamlit UI: 画像アップロード・結果表示・ヒートマップ
requirements.txt	依存パッケージ一覧
models/ (gitignore)	OnnxモデルファイルDL先 (初回起動時に自動取得)
tests/test_face_similarity.py	純粋数学層のユニットテスト (22件)

4. セットアップ・起動方法

以下のコマンドを FaceSimilarity/ ディレクトリ内で実行してください。

```
# ██████████
pip install -r requirements.txt

# ████████ (██████████████████)
streamlit run app.py

# ████████
pytest tests/ -v
```

注意: 初回起動時にONNXモデルが自動ダウンロードされます (合計約38MB)。インターネット接続が必要です。モデルはmodels/ディレクトリに保存され、2回目以降はスキップされます。

5. スコアの解釈

コサイン類似度 $[-1, +1]$ を線形変換して $0 \sim 100\%$ に正規化します。

$-1 \rightarrow 0\%$ (方向が完全に逆) $0 \rightarrow 50\%$ (無相関) $+1 \rightarrow 100\%$ (完全一致)

スコア範囲	判定	SFace論文基準との対応
75% 以上	非常に類似	$\text{cosine} > 0.5$ (高信頼 同一人物相当)
65~74%	比較的類似	$\text{cosine } 0.285 \sim 0.5$ (検証閾値近傍)
50~64%	やや類似	$\text{cosine } 0 \sim 0.285$ (差異あり)
50% 未満	特徴が大きく異なる	$\text{cosine} < 0$ (異なる顔の特徴)

6. ソースコード全文

```
▶ face_similarity.py
from __future__ import annotations

import urllib.request
from dataclasses import dataclass, field
from pathlib import Path

import cv2
import numpy as np

MODELS_DIR = Path(__file__).parent / "models"
_YUNET_FILENAME = "face_detection_yunet_2023mar.onnx"
_SFACE_FILENAME = "face_recognition_sface_2021dec.onnx"
YUNET_PATH = MODELS_DIR / _YUNET_FILENAME
SFACE_PATH = MODELS_DIR / _SFACE_FILENAME

_YUNET_URL = f"https://github.com/opencv/opencv_zoo/raw/main/models/face_detection_yunet/{_YUNET_FILENAME}"
_SFACE_URL = f"https://github.com/opencv/opencv_zoo/raw/main/models/face_recognition_sface/{_SFACE_FILENAME}"

def ensure_models() -> None:
    MODELS_DIR.mkdir(exist_ok=True)
    if not YUNET_PATH.exists():
        print(f"Downloading {_YUNET_FILENAME} ...")
        urllib.request.urlretrieve(_YUNET_URL, str(YUNET_PATH))
    if not SFACE_PATH.exists():
        print(f"Downloading {_SFACE_FILENAME} (~37MB) ...")
        urllib.request.urlretrieve(_SFACE_URL, str(SFACE_PATH))

@dataclass
class PairSimilarity:
    first_index: int
    second_index: int
    score: float

@dataclass
class CommonalityResult:
    image_count: int
    pairs: list[PairSimilarity] = field(default_factory=list)

    @property
    def overall_score(self) -> float:
        if not self.pairs:
            return 0.0
```

```

        return sum(p.score for p in self.pairs) / len(self.pairs)

@property
def most_similar_pair(self) -> PairSimilarity | None:
    return max(self.pairs, key=lambda p: p.score) if self.pairs else None

@property
def least_similar_pair(self) -> PairSimilarity | None:
    return min(self.pairs, key=lambda p: p.score) if self.pairs else None

def cosine_similarity(a: np.ndarray, b: np.ndarray) -> float:
    """Cosine similarity in [-1, 1]. Returns 0.0 for zero or mismatched vectors."""
    a_flat = a.flatten().astype(np.float64)
    b_flat = b.flatten().astype(np.float64)
    if a_flat.shape != b_flat.shape:
        return 0.0
    norm_a = float(np.linalg.norm(a_flat))
    norm_b = float(np.linalg.norm(b_flat))
    if norm_a == 0.0 or norm_b == 0.0:
        return 0.0
    cosine = float(np.dot(a_flat, b_flat) / (norm_a * norm_b))
    return max(-1.0, min(1.0, cosine))

def similarity_score(a: np.ndarray, b: np.ndarray) -> float:
    """Similarity as a percentage [0, 100].
    Maps cosine similarity linearly: -1 -> 0%, 0 -> 50%, +1 -> 100%.
    """
    return (cosine_similarity(a, b) + 1.0) / 2.0 * 100.0

def compute_commonality(embeddings: list[np.ndarray]) -> CommonalityResult:
    """Compute all pairwise similarity scores and derive an overall commonality score."""
    n = len(embeddings)
    pairs: list[PairSimilarity] = []
    for i in range(n):
        for j in range(i + 1, n):
            score = similarity_score(embeddings[i], embeddings[j])
            pairs.append(PairSimilarity(first_index=i, second_index=j, score=score))
    return CommonalityResult(image_count=n, pairs=pairs)

def extract_embedding(image_bgr: np.ndarray) -> np.ndarray | None:
    """Detect the most prominent face and return its 128-dim SFace embedding.

    Returns None if no face is detected in the image.
    """
    ensure_models()
    h, w = image_bgr.shape[:2]

    detector = cv2.FaceDetectorYN.create(
        str(YUNET_PATH), "", (w, h), score_threshold=0.6, nms_threshold=0.3, top_k=5000
    )
    _, faces = detector.detect(image_bgr)
    if faces is None or len(faces) == 0:
        return None

    primary = max(faces, key=lambda f: f[2] * f[3])

    recognizer = cv2.FaceRecognizerSF.create(str(SFACE_PATH), "")
    aligned = recognizer.alignCrop(image_bgr, primary)
    return recognizer.feature(aligned)

def bytes_to_bgr(image_bytes: bytes) -> np.ndarray | None:

```


[illegible]


```

        b = np.array([1, 0, 0], dtype=np.float32)
        assert cosine_similarity(a, b) == 0.0

    def test_mismatched_shape_returns_zero(self):
        a = np.array([1, 0], dtype=np.float32)
        b = np.array([1, 0, 0], dtype=np.float32)
        assert cosine_similarity(a, b) == 0.0

    def test_2d_input_flattened(self):
        a = np.array([[1, 0], [1, 0]], dtype=np.float32)
        b = np.array([1, 0, 1, 0], dtype=np.float32)
        assert cosine_similarity(a, b) == pytest.approx(1.0, abs=1e-6)

    def test_result_clamped_to_minus_one(self):
        a = np.array([1.0], dtype=np.float64)
        b = np.array([-1.0], dtype=np.float64)
        assert cosine_similarity(a, b) >= -1.0

    def test_result_clamped_to_plus_one(self):
        a = np.array([1.0], dtype=np.float64)
        assert cosine_similarity(a, a) <= 1.0

class TestSimilarityScore:
    def test_identical_is_100(self):
        a = np.array([1, 0, 1, 0], dtype=np.float32)
        assert similarity_score(a, a) == pytest.approx(100.0, abs=1e-4)

    def test_opposite_is_0(self):
        a = np.array([1, 0, 1, 0], dtype=np.float32)
        b = np.array([-1, 0, -1, 0], dtype=np.float32)
        assert similarity_score(a, b) == pytest.approx(0.0, abs=1e-4)

    def test_orthogonal_is_50(self):
        a = np.array([1, 0], dtype=np.float32)
        b = np.array([0, 1], dtype=np.float32)
        assert similarity_score(a, b) == pytest.approx(50.0, abs=1e-4)

    def test_known_value_75(self):
        # cosine=0.5 -> score=75
        a = np.array([1, 0, 1, 0], dtype=np.float32)
        b = np.array([1, 0, 0, 1], dtype=np.float32)
        assert similarity_score(a, b) == pytest.approx(75.0, abs=1e-4)

    def test_score_in_range(self):
        rng = np.random.default_rng(42)
        for _ in range(20):
            a = rng.standard_normal(128).astype(np.float32)
            b = rng.standard_normal(128).astype(np.float32)
            s = similarity_score(a, b)
            assert 0.0 <= s <= 100.0

class TestComputeCommonality:
    def _vec(self, *values: float) -> np.ndarray:
        return np.array(values, dtype=np.float32)

    def test_two_identical(self):
        a = self._vec(1, 0, 1, 0)
        result = compute_commonality([a, a])
        assert len(result.pairs) == 1
        assert result.overall_score == pytest.approx(100.0, abs=1e-4)

    def test_three_pairs_count(self):
        a = self._vec(1, 0, 1, 0)

```

```

        b = self._vec(1, 0, 0, 1)
        c = self._vec(-1, 0, -1, 0)
        result = compute_commonality([a, b, c])
        assert len(result.pairs) == 3
        assert result.image_count == 3

    def test_three_pairs_average(self):
        # scores: a-b=75, a-c=0, b-c=25 -> avg=100/3
        a = self._vec(1, 0, 1, 0)
        b = self._vec(1, 0, 0, 1)
        c = self._vec(-1, 0, -1, 0)
        result = compute_commonality([a, b, c])
        assert result.overall_score == pytest.approx(100.0 / 3.0, abs=1e-4)

    def test_three_pairs_indices(self):
        vecs = [self._vec(1, 0), self._vec(0, 1), self._vec(1, 1)]
        result = compute_commonality(vecs)
        indices = {(p.first_index, p.second_index) for p in result.pairs}
        assert indices == {(0, 1), (0, 2), (1, 2)}

    def test_most_similar_pair(self):
        a = self._vec(1, 0, 1, 0)
        b = self._vec(1, 0, 0, 1)
        c = self._vec(-1, 0, -1, 0)
        result = compute_commonality([a, b, c])
        assert result.most_similar_pair is not None
        assert result.most_similar_pair.score == pytest.approx(75.0, abs=1e-4)

    def test_least_similar_pair(self):
        a = self._vec(1, 0, 1, 0)
        b = self._vec(1, 0, 0, 1)
        c = self._vec(-1, 0, -1, 0)
        result = compute_commonality([a, b, c])
        assert result.least_similar_pair is not None
        assert result.least_similar_pair.score == pytest.approx(0.0, abs=1e-4)

    def test_single_image_no_pairs(self):
        result = compute_commonality([self._vec(1, 0)])
        assert result.pairs == []
        assert result.overall_score == 0.0

    def test_empty_list_no_pairs(self):
        result = compute_commonality([])
        assert result.pairs == []
        assert result.overall_score == 0.0
        assert result.most_similar_pair is None
        assert result.least_similar_pair is None

```

7. テスト結果サマリー

☑ 22件 / 22件 全パス (pytest 実行時間: 0.14秒)

テストクラス	件数	内容
TestCosineSimilarity	9	同一・反対・直交・既知値・ゼロ・不一致次元・2D入力・クランプ
TestSimilarityScore	5	100%・0%・50%・既知値75%・ランダム20件の範囲検証
TestComputeCommonality	8	2画像・3ペア数・平均値・インデックス・最大/最小・単一・空リスト

8. 今後の拡張ポイント

- ・複数顔対応: 現在は各画像の最も大きい顔を1つ選択。画像内の全顔を比較対象にする拡張が可能。
- ・グループ化: 類似度閾値を使ったクラスタリング (AgglomerativeClusteringなど) で顔をグループ分け。
- ・「平均顔」生成: 埋め込みベクトルの平均からグループの代表的な顔の特徴を可視化。
- ・バッチ処理: CLIインターフェースを追加してスクリプト連携や自動化パイプラインに対応。
- ・モデル精度向上: ArcFace (insightface) への差し替えでより高精度な識別が可能。

リポジトリ: [diodiodiio/1st_test](#) | ブランチ: [claude/facial-similarity-quantifier-0k6e1v](#) | コミット: [FaceSimilarity/ディレクトリ](#)